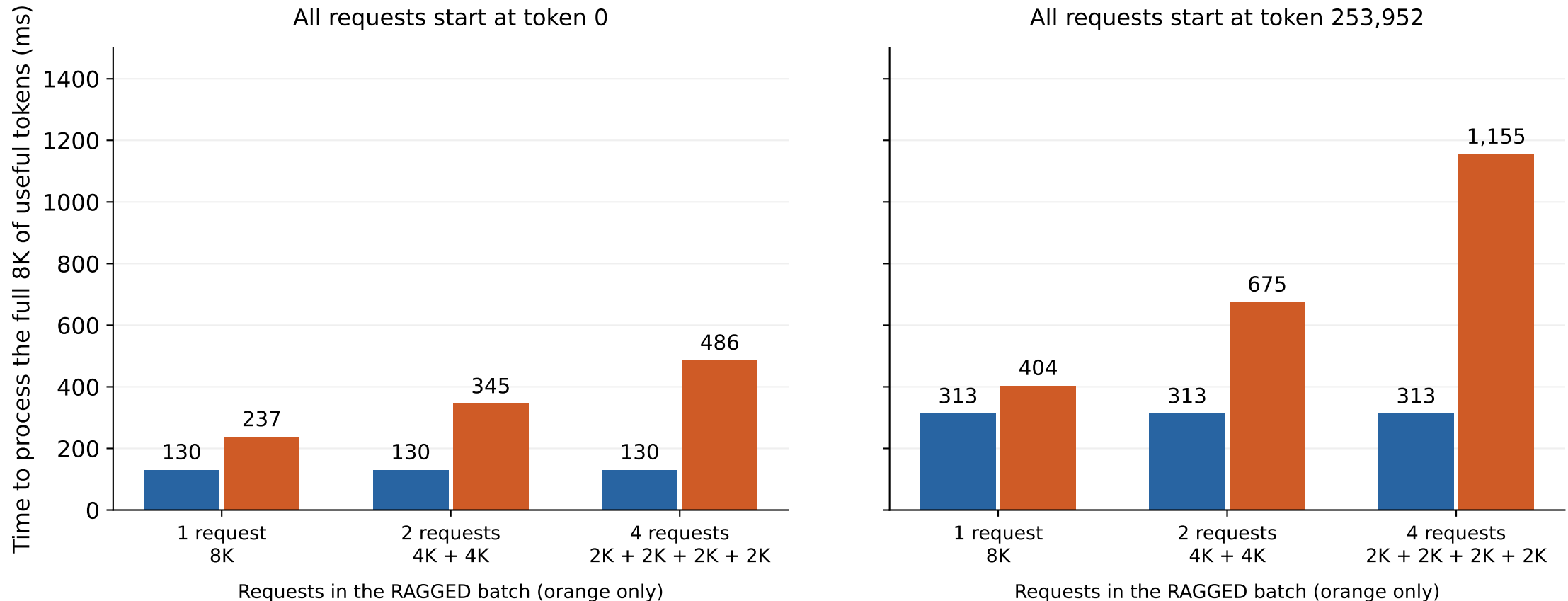


One chunked 8K call vs one full 8K ragged batch

Every bar = ONE call processing 8,192 useful tokens. Lower time is better.

■ Chunked: ONE 8K call, ONE request ■ Ragged: ONE full 8K batch



Question: At the same 8K useful-token budget, how does splitting ragged work across requests affect time?

BLUE ALWAYS means one request processing one full 8K chunk. ORANGE means one packed batch totaling 8K. The request counts on the x-axis apply only to orange. Start index = existing prefix length for each request.

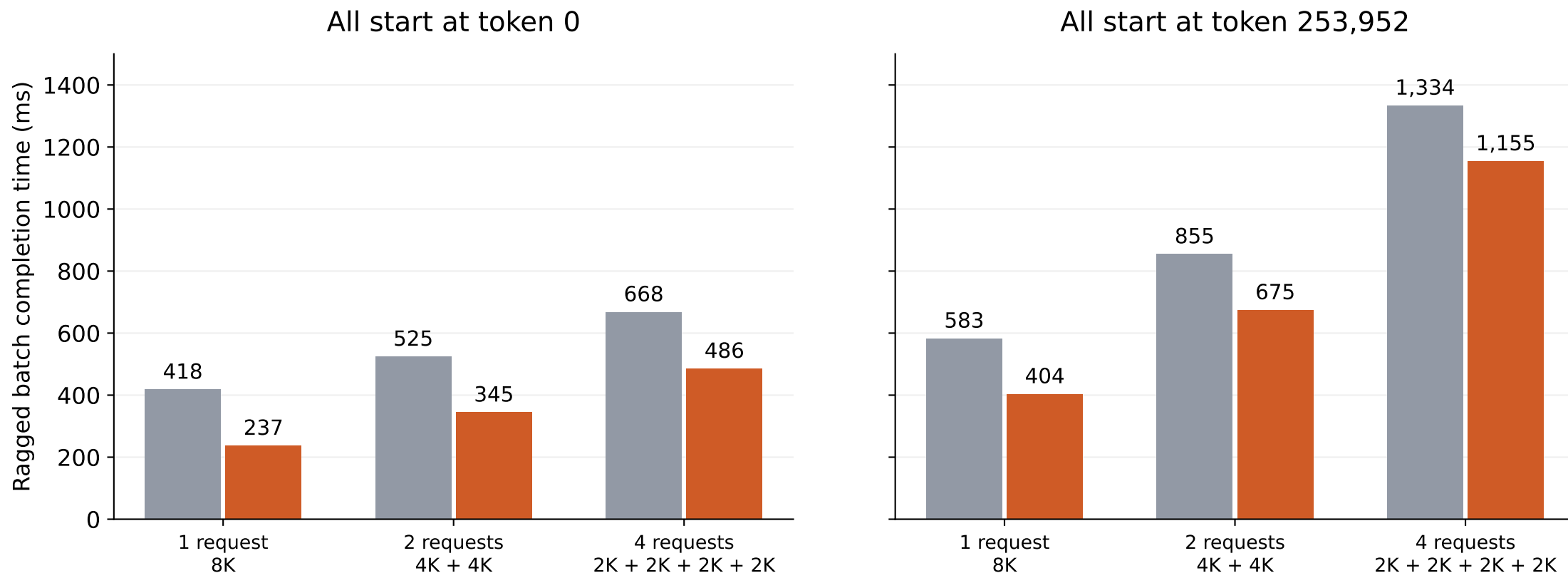
Result: For ragged 4 × 2K versus chunked 1 × 8K: 486 vs 130 ms at the beginning (3.73× the time); 1155 vs 313 ms near the end (3.69× the time).

K = 1,024 tokens. Time completes ONE call (8,192 useful tokens). 60 layers; CP8/TP4; median of five warmed runs including staging and sync. Compilation/capture excluded. Pieces shorter than 8K are final pieces. Both current paths use the original reduce-scatter.

What did removing the FP32 sums save?

Every bar = ONE call processing 8,192 useful tokens. Lower time is better.

■ Before: ragged with FP32 sums ■ Now: ragged with reduce-scatter



Question: How much of ragged prefill's time came from gathering TP partials and adding them in FP32?

Before = previous five-replay medians; now = fresh five-replay medians with the same requests and positions. The unchanged chunked baseline was rerun and differs by at most 1.3% across these cases.

Result: For the first 8K of one request: 418 -> 237 ms, saving 181 ms (43%). Across all 11 cases, savings range from 179 to 182 ms. Packing and attention were unchanged.

K = 1,024 tokens. Time completes ONE call (8,192 useful tokens). 60 layers; CP8/TP4; median of five warmed runs including staging and sync. Compilation/capture excluded. Pieces shorter than 8K are final pieces. Both current paths use the original reduce-scatter.

Numerics: separate 60-layer [8192, 33] batch failed existing tolerance; six-layer check passed. Performance only.